

Remember how back in the day people would

Google themselves, you type your name into a search engine and you see what it knows about you?

Well, the modern equivalent of that is to do the same thing with a chatbot.

So when I ask a large language model, who is Martin Keen?

Well, the response varies greatly depending upon which model I'm asking, because different models, they have different training data sets, they have a different knowledge cutoff dates.

So what a given model knows about me, well, it differs greatly.

But how could we improve the model's answer?

Well, there's three ways.

So let's start with a model here, and we're gonna see how we can improve its responses.

Well, the first thing it could do is it could go out and it could perform a search, a search for new data that either wasn't in its training data set, or it was just data that became available after the model finished training, and then it could incorporate those results from the search back into its answer. That is called RAG or Retrieval Augmented Generation.

That's one method.

Or we could pick a specialized model, a model that's been trained on, let's say, transcripts of these videos.

That would be an example of something called fine tuning,

or we could ask the model a query that better specifies what we're looking for.

So maybe the LLM already knows plenty about the Martin Keens of the world, but let's tell the model that we're referring to the Martin Keen who works at IBM, rather than the Martin Keen that founded Keen Shoes.

That is an example of prompt engineering.

Three ways to get better outputs out of large language models, each with their pluses and minuses.

Let's start with RAG.

So let's break it down.

First there's retrieval.

So retrieval of external up-to-date information.

Then there's augmentation.

That's augmentation of the original prompt with the retrieved information added in.

And then finally there's generation.

That's generation of a response based on all of this enriched context.

So we can think of it like this.

So we start with a query and the query comes in to a large language model.

Now, what RAG is gonna do is it's first going to go searching through a corpus of information.

So we have this corpus here full of some sort of data.

Now, perhaps, that's your organization's documents.

So it might be spreadsheets, PDFs, internal wikis, you know, stuff like that,

But unlike a typical search engine that just matches keywords,

RAG converts both your question, the query, and all of the documents into something

called vector embeddings.
So these are all converted into vectors,
essentially turning words and phrases into long lists of numbers that capture their meaning.
So when you ask a query like, what was our company's revenue growth last quarter?
Well, RAG will find documents that are mathematically similar in meaning to your question,
even if they don't use the exact same words.
So it might find documents mentioning fourth quarter performance or quarterly sales.
Those don't contain the keyword revenue growth, but they are semantically similar.
Now, once RAG finds the relevant information, it adds this information back into your original query before passing it to the language model.
So instead of the model just kind of guessing based on its training data,
it can now generate a response that incorporates your actual facts and figures.
So this makes RAG particularly valuable when you are looking for information that is up to date,
and it's also very valuable when you need to add in information that is domain specific as well,
but there are some costs to this.
Let's go with the red pen.
So one cost, that would be the cost of performance,
for performing all of this, because you have this retrieval step here, and that adds latency to each query compared to a simple prompt to a model.
There are also costs related to just kind of the processing of this as well.
So if we think about what we're having to do here, we've got documents that need to be vector embeddings,
and we need to store these vector embeddings in a database.
All of this adds to processing costs, it adds to infrastructure costs to make this solution work.
All right, next up, fine tuning.
So remember how we discussed getting better answers about me by training a model specifically on, let's say, my video transcripts.
Well, that is fine tuning in action.
So what we do with fine tuning is we take a model, but specifically an existing model, and that existing model has broad knowledge.
And then we're gonna give it additional specialized training on a focused data set.
So this is now specialized to what we want to develop particular expertise on.
Now, during fine tuning, we're updating the model's internal parameters through additional training.
So the model starts out with some weights here,
like this, and those weights were optimized during its initial pre-training.
And as we fine tune, we're making small adjustments here to the model's weights using this specialized data set.
So this is being incorporated.
Now this process typically uses supervised learning where we provide input-output pairs that demonstrate the kind of responses we want.

So for example, if we're fine-tuning for technical support, we might provide thousands of examples of customer queries, and those would be paired with correct technical responses. The model adjusts its weights through back propagation to minimize the difference between its predicted outputs and the targeted responses. So we're not just teaching the model new facts here, we're actually modifying how it processes information. The model is learning to recognize domain-specific patterns. So, fine-tuning shows its strength when you particularly need a model that has very deep domain expertise. That's what we can really add in with fine tuning, and also, it's much faster, specifically at inference time. So when we are putting the queries in, it's faster than RAG because it doesn't need to search through external data, and because the knowledge is kind of baked into the model's weights, you don't need to maintain a separate vector database, but there's some downsides as well. Well, there's certainly issues here with the training complexity of all of this. You're going to need thousands of high quality training examples. There are also issues with computational cost. The computational cost for training this model can be substantial and is going to require a whole bunch of GPUs. And there's also challenges related to maintenance as well because unlike RAG where you can easily add new documents to your knowledge base at any point. Updating a fine-tune model requires another round of training and then perhaps most importantly of all there is a risk of something called catastrophic forgetting. Now that's when the model loses some of its general capabilities while it's busy learning these specialized ones. So finally let's explore prompt engineering. Now specifying Martin Keen who works at IBM versus Martin Keene who founded Keene Shoes, that's prompt engineering, but at its most basic. Prompt engineering goes far beyond simple clarification. So let's think about when we input a prompt, the model receives this prompt and it processes it through a series of layers, and these layers are essentially tension mechanisms and each one focuses on different aspects of your prompt text that came in. And by including specific elements in your prompt, so examples or context or how you want the format to look, you're directing the model's attention to relevant patterns it learned during training. So for example, telling a model to think about this step-by-step, that activates patterns it learnt from training data where methodical reasoning led to accurate results. So a well-engineered prompt can transform a model's output without any additional

training or without data retrieval.

So take an example of a prompt.

Let's say we say, is this code secure?

Not a very good prompt.

An engineered prompt, it might read a bit more like this.

It's much more detailed.

Now.

We haven't changed the model, we haven't added new data, we've just better activated its existing capabilities.

Now I think the benefits to this are pretty obvious.

One is that we don't need to change any of our back-end infrastructure here because there are no infrastructure changes at all in order to prompt better, it's all on the user.

There's also the benefit that by doing this, You get to see immediate responses and immediate results to what you do.

We don't have to add in new training data or any kind of data processing, but of course there are some limitations to this as well.

Prompt engineering is as much an art as it is a science.

So there is certainly a good amount of trial and error in this sort of process to find effective prompts.

and you're also limited in what you can do here, you're limited to existing knowledge because you're not able to actually add anything else in here. No additional amount of prompt engineering is going to teach it truly new information. You're not going to the model anything that's outdated in the model.

So we've talked about now RAG as being one option and we talked about fine tuning as being another one.

And now, just now, we've talked about prompt engineering as well and I've really talked about those as three different distinct things here, but they're commonly used actually in combination.

We might use all three together.

So consider a legal AI system.

RAG, that could retrieve specific cases and recent court decisions.

The prompt engineering part, that could make sure that we follow proper legal document formats by asking for it.

And then fine-tuning, that can help the model master firm-specific policies.

I mean, basically, we can think of it like this.

We can think that prompt engineering offers flexibility and immediate results, but it can't extend knowledge.

RAG, that can extend knowledge, it provides up-to-date information, but with computational overhead.

and then fine-tuning,

that enables deep domain expertise, but it requires significant resources and maintenance.

Basically, it comes down to picking the methods that work for you.

You know, we've, we sure come a long way from vanity searching on Google.

